MERN STACK

TRAINING TR-103 REPORT DAY 19

16 July 2026

Overview:

The nineteenth day of the MERN Stack training focused on React Context API and browser storage mechanisms. I learned how the useContext Hook helps share data across multiple components without prop drilling. The session also covered Local Storage and Session Storage for storing data in the browser. These concepts were applied by creating practical projects, including a Login and Signup Page, to understand state management and data persistence in React.

Learning Objectives:

- Understand the useContext Hook.
- Learn the concept of React Context.
- Understand the problem of prop drilling.
- Learn the difference between Local Storage and Session Storage.
- Build a Login and Signup Page using React.

REACT

useContext Hook

The useContext Hook is a built-in React Hook that allows components to access shared data from a Context without passing props through every level of the component tree.

Features:

- Shares data between multiple components.
- Eliminates prop drilling.
- Simplifies state management.
- Improves code readability.

Context

Context is a React feature that provides a central place to store and share data among components.

Main Parts of Context:

- Context Object – Created using createContext().
- Provider – Supplies data to child components.
- Consumer (useContext) – Accesses the shared data.

Advantages:

- Reduces unnecessary prop passing.
- Makes data sharing easier.
- Improves application structure.

TASK - 1:

Createcon.js :

```
1  import { createContext } from "react"
2
3  export const Createcon = createContext(null)
4
```

Usecon.js

```jsx
import { useContext } from "react"
import { Createcon } from "./createcon"

export const Usecon =()=>{
    const [name,setName] = useContext(Createcon)
    return(
        <>
        <input tyoe ="text" onChange = {(e) =>setName(e.target.value
        {name} This is the first output
        </>
    )
}
```

Usecon2.js

```jsx
import { useContext } from "react"
import { Createcon } from "./createcon"

export const Usecon2 =()=>{
    const [name,setName] = useContext(Createcon)
    return(
        <>
        <input tyoe ="text" onChange = {(e) =>setName(e.target.value
        {name} This is the second output
        </>
    )
}
```

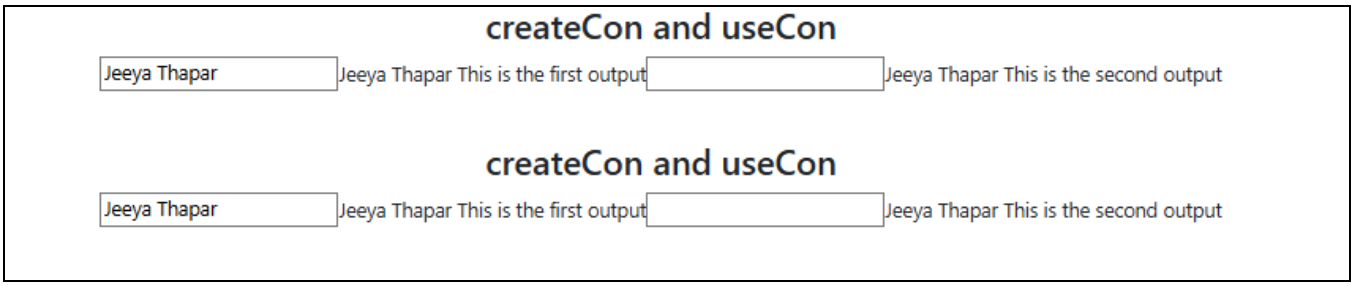
App/js

```jsx
import './App.css';
import {Day1} from './components/Day1'
import { Routes, Route } from "react-router-dom";
import{Navbar} from './components/Navbar';
import {Footer} from './components/Footer';
import {Card} from './components/Card';
import { Home } from "./pages/Home";
import { About } from "./pages/About";
import { Counter } from "./components/Counter";
import { State } from "./components/State";
import { Todos } from './components/todos';
import { useState } from 'react';
import { Createcon } from './components/createcon';
import { Usecon} from './components/usecon';
import { Usecon2} from './components/Usecon2.js';
import { Signup } from './components/Signup.js';
import { Login } from './components/Login.js';

function App() {
  const [name,setName]= useState("")

  return (
    <div className ="App">
        <h3 >createCon and useCon</h3>
        <Createcon.Provider value ={[name,setName]}>
          <Usecon/>
          <Usecon2/>
        </Createcon.Provider>
    </div>
```

Output:



Local Storage

Local Storage is a browser storage mechanism that stores data permanently as key-value pairs. The data remains available even after the browser is closed until it is manually removed.

Features:

- Stores data permanently.
- Saves data as key-value pairs.
- Useful for user preferences and saved information.

Session Storage

Session Storage is used to store data temporarily for a single browser session. The stored data is automatically removed when the browser tab is closed.

Features:

- Stores data temporarily.
- Removes data when the session ends.
- Useful for temporary user information.

Difference Between Local Storage and Session Storage :

Local Storage	Session Storage
Stores data permanently.	Stores data temporarily.
Data remains after the browser is closed.	Data is removed when the browser tab is closed.
Suitable for long-term storage.	Suitable for session-based data.

Login & Signup Page (Task)

A Login and Signup Page was developed using React to understand form handling and browser storage. User information was stored and retrieved using browser storage techniques.

Features:

- User registration.
- User login.
- Form validation.
- Data storage using Local or Session Storage.

TASK - 2:

Signup.js :



```jsx
import { useState } from "react"

export const Signup =({setPage})=>{
    const [name,setName] =useState("");
    const[email,setEmail] = useState("");
    const[password,setPassword]= useState("");

    const handleSignup = ()=>{
        console.log("Button Clicked");
        localStorage.setItem("Name",JSON.stringify(name));
        localStorage.setItem("email",JSON.stringify(email));
        localStorage.setItem("password",JSON.stringify(password));
        alert("SignUp Successfull");
        setPage("login");

    }

   return (
  <div className="container d-flex justify-content-center align-items-center vh-100">
    <div
      className="card shadow p-4"
      style={{ width: "400px", borderRadius: "15px" }}
    >
      <h2 className="text-center mb-4 text-primary">Sign Up</h2>

      <input
        type="text"
        className="form-control mb-3"
        value={name}
        placeholder="Enter your name"
        onChange={(e) => setName(e.target.value)}
      />

      <input
        type="email"
        className="form-control mb-3"
        value={email}
        placeholder="Enter your email"
        onChange={(e) => setEmail(e.target.value)}
      />

      <input
        type="password"
        className="form-control mb-4"
        value={password}
        placeholder="Enter your password"
        onChange={(e) => setPassword(e.target.value)}
      />

      <button
        className="btn btn-primary w-100"
        onClick={handleSignup}
      >
        Sign Up
      </button>
    </div>
  </div>
);
}
```

Login.js :

```jsx
import { useState } from "react"

export const Login =()=>{
    const [name,setName] =useState("");
    const[email,setEmail] = useState("");
    const[password,setPassword]= useState("");

    const handleLogin = ()=>{
        let Name = JSON.parse(localStorage.getItem("Name"));
        let Email= JSON.parse(localStorage.getItem("email"));
        let Password= JSON.parse(localStorage.getItem("password"));

        if(Name ===name && Email === email && Password ===password){
            alert("Login Successful");
        }
        else{
            alert("Invalid creditanials")
        }
    }
 return (
  <div className="container d-flex justify-content-center align-items-center vh-100">
    <div
      className="card shadow p-4"
      style={{ width: "400px", borderRadius: "15px" }}
    >
      <h2 className="text-center mb-4 text-success">Login</h2>

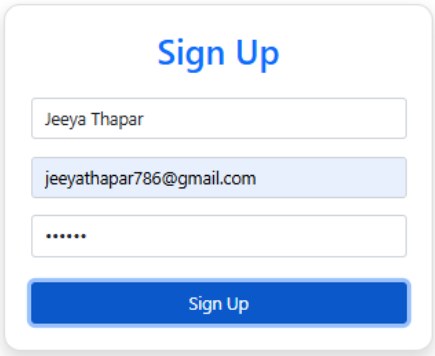
      <input
        type="text"
        className="form-control mb-3"
        value={name}
        placeholder="Enter your name"
        onChange={(e) => setName(e.target.value)}
      />

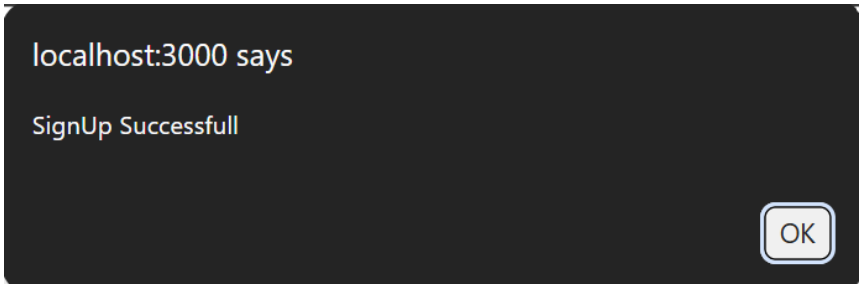
      <input
        type="email"
        className="form-control mb-3"
        value={email}
        placeholder="Enter your email"
        onChange={(e) => setEmail(e.target.value)}
      />

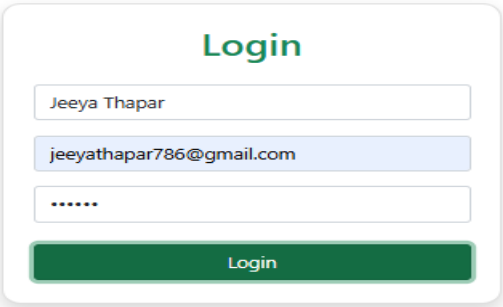
      <input
        type="password"
        className="form-control mb-4"
        value={password}
        placeholder="Enter your password"
        onChange={(e) => setPassword(e.target.value)}
      />

      <button
        className="btn btn-success w-100"
        onClick={handleLogin}
      >
        Login
      </button>
    </div>
</div>
```

Output :


Sign Up
Jeeya Thapar
jeeyathapar786@gmail.com
••••••
Sign Up


localhost:3000 says
SignUp Successfull
OK


Login
Jeeya Thapar
jeeyathapar786@gmail.com
••••••
Login


localhost:3000 says
Login Successful
OK

Conclusion:

By the end of Day 19, I gained a clear understanding of the **`useContext` Hook**, React Context, and browser storage mechanisms. I learned how to share data efficiently between components and store user information using Local Storage and Session Storage while building a Login and Signup application.